סיכום מלא לפי שיעורים

JavaScript • NodeJS • Express • MongoDB • React • Hooks • Tailwind • TypeScript

כתוב בשפה פשוטה, עם אסוציאציות ודוגמאות - ברמה שגם סבתא תוכל להבין

איך להשתמש במסמך: קרא/י קודם את האסוציאציה של כל שיעור, אחר כך את הנקודות החשובות, ואז את "מה לזכור". אם יש קוד, הוא רק דוגמה קטנה שמראה את הרעיון ולא פתרון מלא לכל משימה.

מפת הדרך של הקורס במשפט אחד

הקורס מתחיל ב-JavaScript בסיסי, עובר לעבודה עם דף HTML ודפדפן, ממשיך לאסינכרוניות ושרתים, נכנס למסדי נתונים, ואז עובר לבניית ממשקים מודרניים עם React ולבסוף מוסיף TypeScript כדי שהקוד יהיה מסודר ובטוח יותר.

תוכן עניינים

JavaScript בסיסי ומתקדם

- שיעור 11 - מערכים, ערך מול כתובת, פונקציות חץ ומתודות מערך
- שיעורי בית 11 - מחשבון המרת מטבעות, מערכים ופונקציות חץ
- שיעורי בית 12 - map, forEach, filter ו-spread
- שיעור 13 - אובייקטים, DOM, מחלקות, הורשה ואחסון בדפדפן
- שיעורי בית 13 - DOM, מודעות מכירה וניהול תלמידים
- שיעור 15 - טיפול בשגיאות, Closure, אסינכרוניות, setTimeout, Promise ו-fetch
- חוברת תרגול JavaScript - בניית Task Manager בדפדפן
- שיעור 19 - שיעור חזרה וסיכום כולל JavaScript

Node, Express ו-Database

- שיעור 16 - NodeJS, CLI, npm, מודולים ו-File System
- שיעור 17 - HTTP, URL, REST API, Express, Middleware, Forms ו-fetch POST
- שיעורי בית 18 - NodeJS עמוד כניסה, רישום, אימות ושמירה לקובץ
- שיעור 20 - Database, NoSQL, MongoDB, Mongoose ו-CRUD

React, Hooks, Router, Tailwind ופרויקטים

- שיעור 21 - מבוא ל-React, Vite, Components, JSX, Props, CSS ו-map
- שיעורי בית 22 - React Basics, סטודנטים, Header, Dashboard, Footer ו-AddStudent
- שיעור 22 - Hooks, useState, Immutable מול Mutable, onChange והעברת פונקציות ב-props
- שיעור 23 - React Router, Link, useNavigate, Context API, useContext ו-useParams
- שיעורי בית 23 - Smart House: חדרים, מוצרים, ניתוב ועיצוב עם AI
- שיעור 24 - useEffect, Fetching Data, useMemo ו-useRef
- שיעורי בית 24 - אפליקציית סינון מוצרים עם API, useEffect ו-useRef
- שיעור 25 - Tailwind CSS
- פרויקט 25 - אתר דירוג סרטים

AI, TypeScript ו-React עם TypeScript

- שימוש נכון ב-AI בפיתוח
- שיעור 26 - TypeScript בסיסי: טיפוסים, Objects, Functions, Enum, Tuple, tsconfig, Compiler ו-Type
- שיעור 26 חלק 2 - TypeScript עם React: Props ו-State
- שיעורי בית 1 TS - ניהול ספרים וניהול פרופיל משתמש
- שיעורי בית 2 React + TypeScript - אפליקציית ניהול תקציב

JavaScript בסיסי ומתקדם

1. שיעור 11 - מערכים, ערך מול כתובת, פונקציות חץ ומתודות מערך

מקור: Lesson 11.pptx.pdf

אסוציאציה קלה: דמייני מגירה עם תאים ממוספרים. כל תא מכיל משהו. זה מערך. כשמעבירים תפוח מתא לתא זה ערך; כשנותנים למישהו את המפתח למגירה כולה, זו כתובת בזיכרון.

מה למדנו כאן?

- By Value: מספרים, מחרוזות ובוליאנים עוברים כמו צילום של דף. אם משנים את הצילום, המקור לא משתנה.
- By Reference: מערכים, אובייקטים ופונקציות עוברים כמו כתובת של בית. אם שני אנשים מחזיקים אותה כתובת ומשנים את הבית, כולם רואים את השינוי.
- מערך הוא רשימה מסודרת. לכל פריט יש אינדקס שמתחיל מ-0, כלומר הפריט הראשון נמצא במקום 0 ולא 1.
- push ו-pop עובדים על סוף המערך: push מוסיף לסוף, pop מוציא מהסוף.
- unshift ו-shift עובדים על תחילת המערך: unshift מוסיף להתחלה, shift מוציא מהתחלה.
- sort ממיין. במספרים צריך לתת פונקציית השוואה, אחרת JavaScript עלולה למיין כאילו המספרים הם טקסט.
- splice הוא כמו מספריים ודבק: אפשר לחתוך פריטים מהאמצע ואפשר להדביק פריטים חדשים בדיוק באותו מקום.
- let חי בתוך האזור שבו יצרנו אותו, var פחות מוגבל ולכן יכול לגרום להפתעות. עדיף בדרך כלל let או const.
- Arrow Function היא דרך קצרה יותר לכתוב פונקציה, בעיקר כשמשתמשים בפונקציות בתוך map, filter, forEach וכדומה.
- forEach עובר על כל פריט ולא בונה מערך חדש. map עובר על כל פריט ומחזיר מערך חדש. filter מסנן ומחזיר רק את מה שעבר תנאי. find מחזיר את הראשון שמצא. reduce מצמצם מערך לערך אחד, למשל סכום.
- spread (...) הוא כמו לפתוח שקית ולשפוך את כל הפריטים שלה לתוך שקית חדשה.

מה חשוב לזכור?

- מערכים ואובייקטים לא מעתיקים ערך פשוט; לרוב הם מחזיקים כתובת.
- כאשר רוצים לא להרוס את המקור, יוצרים עותק: const copy = [...arr].
- map מחזיר מערך חדש, forEach לא.
- filter לא מוחק מהמערך המקורי, אלא מחזיר מערך מסונן חדש.
- sort למספרים: arr.sort((a, b) => a - b).

דוגמה קטנה בקוד

```
const nums = [1, 5, 2, 9];
const sorted = [...nums].sort((a, b) => a - b); // [1, 2, 5, 9]
const doubled = nums.map(num => num * 2);      // [2, 10, 4, 18]
const big = nums.filter(num => num > 3);       // [5, 9]
const sum = nums.reduce((total, num) => total + num, 0); // 17
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

2. שיעורי בית 11 - מחשבון המרת מטבעות, מערכים ופונקציות חץ

מקור: שיעורי בית 11.docx.pdf

אסוציאציה קלה: זה כמו לבנות קופה קטנה שממירה שקלים לדולרים, ובמקביל להתאמן על סידור רשימת קניות, מציאת כפילויות וספירת פריטים.

מה למדנו כאן?

- המשימה הגדולה: ליצור מחשבון להמרת מטבעות עם דגש על UI/UX, כלומר שהמסך יהיה יפה, ברור ונעים לשימוש.
- צריך להשתמש ב-AI כדי לשפר את הממשק, אך עדיין להבין את הקוד ולא להעתיק בעיניים עצומות.
- בסוף צריך להעלות את הפרויקט ל-GitHub ול-Vercel, כלומר גם לשמור קוד וגם לפרסם אתר באינטרנט.
- תרגילי מערכים כוללים: מיון מספרים מהגדול לקטן, עבודה עם שני מערכים, מציאת ערכים משותפים בלי includes, וספירת כמה פעמים מספר מופיע במערך.
- תרגילי Arrow Function כוללים פונקציות קצרות שמחזירות תשובה בוליאנית, למשל האם מחרוזת ארוכה מ-5 תווים.
- התרגיל מלמד לחשוב גם על לוגיקה וגם על חוויית משתמש. לא מספיק שהקוד עובד; המשתמש צריך להבין מה לעשות.

מה חשוב לזכור?

- UI הוא איך זה נראה. UX הוא כמה קל ונעים להשתמש בזה.
- לא להשתמש ב-includes בתרגיל שבו נאסר. במקרה כזה עוברים ידנית בלולאות.
- כשמעלים ל-Vercel, חשוב לוודא שהפרויקט עובד גם אחרי build.
- פונקציה שמחזירה true/false נקראת פונקציה בוליאנית.

דוגמה קטנה בקוד

```
const isLongText = text => text.length > 5;

function countNumber(arr, wanted) {
  let count = 0;
  for (let num of arr) {
    if (num === wanted) count++;
  }
  return count;
}
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

3. שיעורי בית 12 - map, forEach, filter ו-spread

מקור: שיעורי בית 12.docx.pdf

> אסוציאציה קלה: דמייני שאת עוברת על סל כביסה: לפעמים את רק מסתכלת על כל בגד, לפעמים את מסננת רק לבנים, ולפעמים את יוצרת ערימה חדשה עם שינוי בכל פריט.

מה למדנו כאן?

- forEach מתאים כשצריך לעבור על כל איבר ולעשות פעולה, כמו להדפיס למסך.
- map מתאים כשצריך ליצור מערך חדש באותו אורך, אבל עם ערכים שעברו שינוי.
- filter מתאים כשצריך ליצור מערך חדש עם רק חלק מהאיברים.
- spread מתאים כשצריך להעתיק מערך או לצרף מערכים בלי לגעת ישירות במקור.
- התרגילים כוללים בדיקת מספרים שמתחלקים ב-3, המרת אותיות גדולות וקטנות לסימונים, טיפול באינדקסים, סינון גילאים מעל 18, יצירת מערך בלי אינדקס מסוים, והוספת שמות למערך חיצוני.
- הרעיון המרכזי: במקום לשנות ידנית כל פריט, משתמשים בכלים מוכנים שעוברים על הרשימה בצורה נקייה.

מה חשוב לזכור?

- forEach = עושה פעולה לכל פריט, לא מחזיר מערך חדש.
- map = מחזיר מערך חדש אחרי שינוי.
- filter = מחזיר מערך חדש אחרי סינון.
- spread = מעתיק או מחבר מערכים בצורה נקייה.

- כשמבקשים לא לכלול אינדקס מסוים, filter עם הפרמטר index יכול לעזור.

דוגמה קטנה בקוד

```
const ages = [12, 18, 22, 30, 15];
const adults = ages.filter(age => age >= 18);

const nums = [2, 4, 1, 2, 7, 2, 8];
const withoutIndex3 = nums.filter((item, index) => index !== 3);
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

4. שיעור 13 - אובייקטים, DOM, מחלקות, הורשה ואחסון בדפדפן

מקור: Lesson 13.pptx.pdf

אסוציאציה קלה: אובייקט הוא כרטיסיה של אדם: שם, משפחה, עיר וציון. DOM הוא הדרך שבה JavaScript נוגע בדף. Class הוא תבנית להכנת הרבה כרטיסיות דומות.

מה למדנו כאן?

- אובייקט שומר כמה פרטים תחת שם אחד. במקום הרבה משתנים נפרדים, שמים אותם בתוך קופסה אחת עם מפתחות וערכים.
- גישה למאפיין נעשית עם נקודה: student.firstName. אפשר לקרוא ערך ואפשר לשנות אותו.
- DOM הוא המפה של דף ה-HTML. דרכו JavaScript יכול למצוא כותרת, תמונה, כפתור או רשימה ולשנות אותם.
- document.getElementById מחפש אלמנט לפי id. יש גם חיפוש לפי class או tag.
- innerHTML משנה תוכן של אלמנט. style משנה עיצוב. createElement, appendChild ו-removeChild יוצרים, מוסיפים ומוחקים אלמנטים.
- Class היא תבנית ליצירת אובייקטים. constructor הוא כמו טופס שחייבים למלא כשמייצרים אובייקט חדש.
- Methods הן פעולות ששייכות למחלקה, למשל discount במוצר או getFullName בסטודנט.
- Inheritance, הורשה, מאפשרת למחלקה אחת לקבל תכונות ומתודות ממחלקה אחרת, כדי לא לכתוב אותו קוד שוב ושוב.
- localStorage ו-sessionStorage שומרים מידע בדפדפן. localStorage נשאר גם אחרי שסוגרים את הדפדפן. sessionStorage נשאר רק עד שסוגרים את הלשונית או הדפדפן.
- תרגול השיעור כולל שינוי תמונה בלחיצת כפתור, שימוש בקישור מתוך input, בניית Student class, ושמירת שם משתמש בדפדפן.

מה חשוב לזכור?

- Object = קופסה עם תכונות.
- DOM = הגשר בין JavaScript למסך.
- Class = מפעל לייצור אובייקטים דומים.
- constructor = מה חייבים לתת כדי ליצור אובייקט.
- localStorage שומר לטווח ארוך, sessionStorage לטווח קצר.

דוגמה קטנה בקוד

```
class Student {
  constructor(firstName, lastName, grades) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.grades = grades;
  }

  getFullName() {
    return `${this.firstName} ${this.lastName}`;
```

```
  }
}
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

5. שיעורי בית 13 - DOM, מודעות מכירה וניהול תלמידים

מקור: שיעורי בית 13.docx.pdf

אסוציאציה קלה: כאן מתרגלים לקחת מידע ולהציג אותו על המקרר של הבית. המשתמש מקליד, והאתר מחליט מה להראות לו.

מה למדנו כאן?

- תרגיל ראשון: פונקציה שמקבלת שם פרטי ומשפחה ומציגה אותם ב-DIVים על הדף.
- תרגיל שני: אתר מודעות מכירה. יש מערך של מוצרים, ולמשתמש יש input אחד שבו הוא מקליד מחיר. בכל שינוי מחיר מציגים מוצרים רלוונטיים.
- הדגש הוא להשתמש באירוע onInput ולא בלחיצה על כפתור. כלומר האתר מגיב תוך כדי הקלדה.
- תרגיל שלישי: מערכת קטנה לניהול תלמידים. יוצרים מערך אובייקטים של תלמידים ומציגים אותם יפה ב-HTML.
- נדרשת גם עבודת עיצוב, אפשר בעזרת AI, אבל חשוב להבין שהעיצוב לא מחליף את הלוגיקה.
- המשימה מחברת בין אובייקטים, מערכים, DOM ואירועים.

מה חשוב לזכור?

- onInput מופעל בכל שינוי בשדה, גם בלי כפתור.
- מערך של אובייקטים הוא מבנה מצוין לרשימות כמו תלמידים או מוצרים.
- כדי להציג רשימה במסך, יוצרים HTML דינמי או אלמנטים חדשים בלולאה.
- עיצוב טוב עוזר להבין את הנתונים, לא רק לעשות יפה.

דוגמה קטנה בקוד

```
const products = [
  { name: 'Chair', price: 100 },
  { name: 'Table', price: 250 }
];

function showByPrice(maxPrice) {
  return products.filter(product => product.price <= maxPrice);
}
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

6. שיעור 15 - טיפול בשגיאות, Closure, אסינכרוניות, setTimeout, Promise ו-fetch

מקור: lesson 15.pptx.pdf

אסוציאציה קלה: זה שיעור על מה עושים כשמשהו משתבש, ואיך לא מחכים ליד התנור עד שהעוגה מוכנה. שמים טיימר וממשיכים לעשות דברים אחרים.

מה למדנו כאן?

- try/catch מאפשרים להריץ קוד מסוכן בלי להפיל את כל התוכנית. אם יש תקלה, catch תופס אותה ומציג הודעה או טיפול מתאים.
- throw מאפשר לזרוק שגיאה בעצמנו. למשל אם המשתמש השאיר שדה ריק, אנחנו מחליטים שזו שגיאה.
- Closure הוא מצב שבו פונקציה זוכרת משתנים מהסביבה שבה נוצרה, גם אחרי שהסביבה הזו כבר הסתיימה. זה שימושי למשתנים פרטיים.

- סינכרוני אומר פעולה אחרי פעולה, כמו תור בקופה. אסינכרוני אומר שאפשר לשלוח פעולה ארוכה לרקע ולהמשיך בינתיים, כמו מלצר שמעביר הזמנה למטבח וחוזר ללקוחות אחרים.
- setTimeout מריץ קוד אחרי זמן מסוים. הוא לא עוצר את כל הקוד, אלא מתזמן פעולה לעתיד.
- Promise הוא הבטחה לתוצאה עתידית: הצלחה עם resolve או כישלון עם reject.
- fetch הוא Promise שמבצע בקשה לשרת. הוא משמש לקבלת נתונים ממקור חיצוני.
- הדגש המעשי: קוד שמדבר עם שרת לא מקבל תשובה מיד, ולכן צריך לדעת לטפל בהמתנה ובשגיאות.

מה חשוב לזכור?

- try/catch = רשת ביטחון.
- throw = אני יוצר שגיאה בעצמי.
- setTimeout = טיימר.
- Promise = תשובה שתגיע בעתיד.
- fetch = ללכת להביא מידע מהאינטרנט או מהשרת.

דוגמה קטנה בקוד

```
try {
  if (!userName) {
    throw new Error('Name is required');
  }
} catch (error) {
  console.log(error.message);
}

setTimeout(() => {
  console.log('This runs later');
}, 2000);
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

7. חוברת תרגול JavaScript - בניית Task Manager בדפדפן

מקור: חוברת תרגול JavaScript.pdf

> אסוציאציה קלה: בונים פנקס משימות קטן. בהתחלה רק כותבים כמה משימות יש, אחר כך מציגים אותן, מוסיפים כפתורים, שומרים בדפדפן, ולבסוף מביאים משימות מבחוץ.

מה למדנו כאן?

- החוברת בנויה בשלבים. כל שלב מוסיף יכולת חדשה לאפליקציית Task Manager פשוטה בלי ספריות ובלי עיצוב מורכב.
- חלק 1: משתנים ו-Data Types. לומדים לשמור מידע בזיכרון: מספר משימות, שם משתמש, מצב התחברות.
- חלק 2: תנאים. לומדים להגיד למחשב מה לעשות לפי מצב: אין משימות, מעט משימות או הרבה משימות.
- חלק 3: מערכים. יוצרים רשימת משימות, מוסיפים משימה, מדפיסים את כולן בלולאה.
- חלק 4: אובייקטים. כל משימה נהיית אובייקט עם title ו-completed, כלומר שם והאם הושלמה.
- חלק 5: פונקציות. מסדרים את הקוד לפעולות חוזרות: addTask, printTasks, countCompleted.
- חלק 6: DOM. מפסיקים לעבוד רק בקונסול ומתחילים להציג משימות במסך עם ul ו-li.
- חלק 7: Events. מוסיפים תגובה ללחיצות: כפתור הוספה וסימון משימה כהושלמה.
- חלק 8: Storage. שומרים את המשימות ב-localStorage כדי שלא ייעלמו ברענון הדף.
- חלק 9: Errors ו-Debugging. לומדים לתפוס שגיאות, למשל משימה בלי שם, ולהציג הודעה ידידותית.
- חלק 10: API ואסינכרוניות. משתמשים ב-fetch כדי להביא משימות מ-jsonplaceholder, משלבים אותן במערכת, משתמשים ב-async/await, מטפלים ב-loading ובשגיאות.
- האתגר בסוף מחבר הכל: בלחיצת כפתור טוענים משימות מה-API, מציגים Loading, מטפלים בשגיאה ומשלבים את הנתונים במערכת.

מה חשוב לזכור?

- Task Manager הוא פרויקט לימודי מושלם כי יש בו כמעט את כל יסודות JS.
- כל משימה צריכה להיות אובייקט, לא רק טקסט, כדי שאפשר יהיה לשמור עליה מידע נוסף.
- לפני שמציירים מחדש רשימה ב-DOM, מנקים את הרשימה הישנה כדי לא לשכפל פריטים.
- localStorage שומר מחרוזות, לכן משתמשים ב-JSON.stringify וב-JSON.parse.
- ב-fetch תמיד כדאי לטפל גם במצב טעינה וגם במצב כישלון.

דוגמה קטנה בקוד

```
const tasks = [{ title: 'Clean room', completed: false }];

function saveTasks() {
  localStorage.setItem('tasks', JSON.stringify(tasks));
}

function loadTasks() {
  return JSON.parse(localStorage.getItem('tasks')) || [];
}
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

8. שיעור 19 - שיעור חזרה וסיכום כולל JavaScript

מקור: lesson 19.pdf

> אסוציאציה קלה: זה כמו שיעור סיכום לפני מבחן נהיגה: עוברים על ההגה, הבלמים, המראות, החניה והכביש המהיר. לא לומדים דבר אחד חדש, אלא מחברים הכל.

מה למדנו כאן?

- הוספת JavaScript ל-HTML יכולה להיות בתוך הקובץ או בקובץ חיצוני. עדיף לרוב קובץ חיצוני כי הקוד מסודר יותר.
- הדפסה ותצוגה: console.log למפתח, alert למשתמש, ושינוי DOM כדי להציג מידע בדף עצמו.
- Data Types: משתנים, let/const/var, שמות משמעותיים ב-camelCase, פעולות מתמטיות ופעולות על מחרוזות.
- תנאים: if/else, switch/case ותנאים מקוצרים. המטרה היא לגרום לקוד לקבל החלטות.
- כלי פיתוח: console, debugger, Network לבדיקת קריאות, Application לבדיקת storage.
- פונקציות: קוד שמקבל ערכים, מחזיר ערכים, נקרא מתוך HTML, ואפשר לכתוב אותו גם כ- arrow function.
- לולאות: while, for, do while, break לעצירה ו-continue לדילוג.
- מערכים: forEach, filter, map, find, reduce ו-spread.
- אובייקטים: מבנה של key/value, אובייקט מקונן, מערך אובייקטים, גישה לערכים ו-destructuring.
- DOM: מציאת אלמנטים, שינוי אלמנטים והוספת פרטים לדף.
- Class: תבניות לאובייקטים, מתודות והורשה.
- שגיאות: try/catch, throw ואובייקט Error.
- Storage: localStorage, sessionStorage ובונוס cookies.
- נושאים מתקדמים: closure, סינכרוני מול אסינכרוני, promises, fetch, scoping, events, hoisting ו- event loop.

מה חשוב לזכור?

- זה שיעור שמטרתו לראות את התמונה הגדולה של JS.
- כמעט כל אפליקציית JS מתחילה במידע, תנאים, לולאות ופונקציות, ואז עוברת ל-DOM ואירועים.
- אחרי שמבינים אסינכרוניות ו-fetch, אפשר להתחיל לדבר עם שרתים ו-APIs.
- כלי הפיתוח בדפדפן הם חלק מהעבודה, לא משהו צדדי.

דוגמה קטנה בקוד

```
// destructuring example
const user = { name: 'Dana', age: 30 };
const { name, age } = user;

// event example
button.addEventListener('click', () => {
  console.log('Clicked');
});
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

Database-ו Node, Express

9. שיעור 16 - NodeJS, CLI, npm, מודולים ו-File System

מקור: Lesson 16.pptx.pdf

אסוציאציה קלה: עד עכשיו JavaScript עבד בתוך הדפדפן. NodeJS הוא כמו לתת ל-JavaScript מפתח למחסן האחורי: הוא יכול לרוץ במחשב, לקרוא קבצים, לכתוב קבצים ולהפעיל שרתים.

מה למדנו כאן?

- NodeJS היא סביבת עבודה שמריצה JavaScript מחוץ לדפדפן, בעיקר בצד שרת.
- NodeJS מבוססת על מנוע V8 של Chrome, שמתרגם JavaScript לשפת מכונה.
- היתרון הגדול: עבודה אסינכרונית. Node יכול לטפל בהרבה בקשות בלי לחכות שכל פעולה תסתיים לפני שהוא ממשיך.
- CLI הוא חלון הפקודות. דרכו יוצרים תיקיות, קבצים, מריצים node ומתקינים חבילות.
- npm הוא מנהל החבילות. דרכו מתקינים ספריות מוכנות שאנשים אחרים כתבו.
- package.json הוא תעודת הזהות של הפרויקט: שם, גרסה, קובץ כניסה, סקריפטים ותלויות.
- require מייבא מודול. module.exports מייצא פונקציה או ערך מקובץ לקובץ אחר.
- JSON הוא פורמט טקסטואלי של key/value, נוח להעברת מידע בין מערכות.
- File System, או fs, מאפשר לעבוד עם קבצים: ליצור, לכתוב, להוסיף, לקרוא, למחוק ולשנות שם.
- התרגולים כוללים התקנת חבילות superheroes ו-supervillains, יצירת מודול מחשבון, כתיבה לקובץ aboutMe.txt, וקריאת קובץ כדי לספור מילים.

מה חשוב לזכור?

- NodeJS = JavaScript מחוץ לדפדפן.
- npm install = להביא ספרייה לפרויקט.
- require = להשתמש בספרייה או בקובץ אחר.
- fs.writeFile כותב מחדש, fs.appendFile מוסיף לסוף.
- package.json מנהל את הפרויקט.

דוגמה קטנה בקוד

```
const fs = require('fs');

fs.writeFile('aboutMe.txt', 'My name is Dana', (err) => {
  if (err) throw err;
  console.log('File saved');
});
```

סיכום של סבתא

כאן עוברים מאתר שרק רואים בדפדפן למערכת אמיתית עם שרת, בקשות, תשובות ושמירת מידע. זה הצד האחורי שמאפשר לאפליקציה לזכור ולעבוד עם משתמשים.

10. שיעור 17 - HTTP, URL, REST API, Express, Middleware, Forms ו-fetch POST

מקור: Lesson 17.pptx.pdf

אסוציאציה קלה: האינטרנט הוא כמו דואר: הדפדפן שולח בקשה לשרת, והשרת מחזיר תשובה. Express הוא הפקיד שמסדר את הדואר ומחליט מה להחזיר לכל כתובת.

מה למדנו כאן?

- HTTP הוא פרוטוקול תקשורת בין לקוח לשרת. הלקוח מבקש, השרת מחזיר תגובה.
- URL הוא הכתובת. הוא כולל פרוטוקול, דומיין ונתיב. לדוגמה נתיב /food מבקש אזור מסוים באתר.
- REST API הוא דרך לבנות שירותים לפי פעולות CRUD: יצירה, קריאה, עדכון ומחיקה.
- Express היא ספרייה פופולרית לבניית שרתים ב-NodeJS.
- app.get מגדיר תגובה לבקשת GET, כלומר בקשה לקבל מידע.
- app.listen מפעיל את השרת על פורט מסוים, למשל 3000.

- Middleware היא פונקציה באמצע הדרך. היא יכולה לבדוק, לשנות או לעצור בקשות לפני שהן מגיעות ליעד.
- קבצים סטטיים הם HTML, CSS, תמונות וקבצים שלא נוצרים מחדש בכל בקשה. Express יכול להגיש אותם מתיקייה.
- body-parser היה כלי לניתוח מידע שנשלח בטופס או JSON; כיום Express כולל יכולות כאלה מובנות.
- טפסים יכולים לשלוח מידע לשרת ב-POST. אם רוצים לשלוח JSON עם fetch, משתמשים ב-event.preventDefault כדי לעצור את התנהגות ברירת המחדל של הטופס.
- Query Parameters הם דרך לשלוח מידע בתוך ה-URL, למשל ?name=dana.
- Status Codes הם מספרי תשובה מהשרת: 2XX הצלחה, 4XX שגיאת משתמש, 5XX שגיאת שרת.

מה חשוב לזכור?

- GET = להביא מידע. POST = לשלוח מידע חדש.
- Express = הדרך הנוחה ליצור שרת ב-Node.
- Middleware = תחנת בדיקה בדרך.
- req הוא מה שהלקוח שלח. res הוא מה שהשרת מחזיר.
- event.preventDefault מונע מהטופס לרענן את הדף כששולחים עם fetch.

דוגמה קטנה בקוד

```
const express = require('express');
const app = express();

app.use(express.json());

app.get('/', (req, res) => {
  res.send('Hello world');
});

app.listen(3000, () => console.log('Server is running'));
```

סיכום של סבתא

כאן עוברים מאתר שרק רואים בדפדפן למערכת אמיתית עם שרת, בקשות, תשובות ושמירת מידע. זה הצד האחורי שמאפשר לאפליקציה לזכור ולעבוד עם משתמשים.

11. שיעורי בית NodeJS - 18 עמוד כניסה, רישום, אימות ושמירה לקובץ

מקור: שיעורי בית docx.pdf.18

אסוציאציה קלה: זה כמו טופס הרשמה למועדון: קודם שואלים שם, אחר כך ממלאים שם משתמש, אימייל וסיסמה. אם הכל תקין, כותבים את הפרטים במחברת של המועדון.

מה למדנו כאן?

- צריך ליצור עמוד כניסה לאפליקציה עם שדה שם. השם חייב להיות באורך של לפחות 2 תווים.
- לחיצה על כפתור מעבירה לעמוד רישום בנתיב signup/.
- עמוד הרישום נבנה עם form ומשתמש ב-GET ו-POST.
- שם משתמש חייב להיות בין 4 ל-8 תווים.
- כתובת אימייל חייבת להכיל @.
- סיסמה חייבת להיות בין 5 ל-10 תווים ולכלול את התו $.
- אישור סיסמה חייב להיות זהה לסיסמה.
- אם הכל תקין, המשתמש עובר לעמוד בית שמציג hello Username.
- אם אחד השדות לא מולא כראוי, יש להציג Error.
- בשרת צריך לשמור את כל פרטי המשתמש בקובץ טקסט.

מה חשוב לזכור?

- ולידציה בשרת חשובה גם אם יש ולידציה בצד לקוח.
- POST מתאים לשליחת טופס עם מידע רגיש יותר מ-GET.
- שמירה לקובץ נעשית עם fs, לרוב appendFile כדי לא למחוק משתמשים קודמים.

- אחרי הרשמה מוצלחת אפשר לבצע redirect לעמוד בית.

דוגמה קטנה בקוד

```
app.post('/signup', (req, res) => {
  const { username, email, password, confirm } = req.body;
  if (password !== confirm) return res.status(400).send('Error');
  // save to text file here
  res.redirect('/home');
});
```

סיכום של סבתא

כאן עוברים מאתר שרק רואים בדפדפן למערכת אמיתית עם שרת, בקשות, תשובות ושמירת מידע. זה הצד האחורי שמאפשר לאפליקציה לזכור ולעבוד עם משתמשים.

12. שיעור 20 - Database, NoSQL, MongoDB, Mongoose ו-CRUD

מקור: Lesson 20.pptx.pdf

אסוציאציה קלה: מסד נתונים הוא ארון מסודר. ב-SQL הארון בנוי מטבלאות קשיחות. ב-NoSQL, כמו MongoDB, יש קלסרים עם דפים גמישים שנראים כמו אובייקטים.

מה למדנו כאן?

- Database הוא מקום מסודר לאחסון מידע, שליפה, עדכון ומחיקה.
- SQL עובד בטבלאות עם יחסים. NoSQL עובד בצורה גמישה יותר, לרוב עם אוספים ומסמכים.
- MongoDB הוא מסד NoSQL ששומר נתונים כ-documents דמויי JSON.
- Database יכול להכיל כמה collections. כל collection מכיל documents. כל document הוא אובייקט עם מאפיינים וערכים.
- MongoDB Atlas הוא שירות ענן שמאפשר לשמור מסד נתונים באינטרנט ולגשת אליו ממקומות שונים.
- MongoDB for VS Code מאפשר לראות את הנתונים מתוך VS Code.
- Mongoose היא ספרייה שמחברת בין NodeJS ל-MongoDB בצורה מסודרת.
- Schema מגדיר את תבנית הנתונים. Model הוא הכלי שבעזרתו עובדים בפועל מול ה-collection.
- insertOne ו-insertMany מוסיפים מסמך אחד או כמה מסמכים.
- find מחזיר את כל ההתאמות. findOne מחזיר התאמה אחת ראשונה.
- findOneAndUpdate ו-updateMany מעדכנים נתונים.
- deleteOne ו-deleteMany מוחקים נתונים.
- Comparison Query Operators כמו $gte, $lt, $lte, gt עוזרים לחפש לפי גדול מ, קטן מ וכדומה.
- התרגול כולל מאגר השכרת רכבים, הכנסת רכבים, הצגת רכבים שחורים והצגת רכבים משנתון 2020 ומעלה.

מה חשוב לזכור?

- MongoDB = מסד נתונים של אובייקטים גמישים.
- Collection = תיקייה של מסמכים דומים.
- Document = אובייקט אחד שנשמר במסד.
- Schema = התבנית. Model = כלי העבודה.
- CRUD = Create, Read, Update, Delete.

דוגמה קטנה בקוד

```
const carSchema = new mongoose.Schema({
  plateNumber: String,
  model: String,
  year: Number,
  color: String
});

const Car = mongoose.model('Car', carSchema);
const newCars = await Car.find({ year: { $gte: 2020 } });
```

סיכום של סבתא

כאן עוברים מאתר שרק רואים בדפדפן למערכת אמיתית עם שרת, בקשות, תשובות ושמירת מידע. זה הצד האחורי שמאפשר לאפליקציה לזכור ולעבוד עם משתמשים.

React, Hooks, Router, Tailwind ופרויקטים

13. שיעור 21 - מבוא ל-CSS, Props, JSX, Components, Vite, React ו-map

מקור: Lesson 21.pptx.pdf

אסוציאציה קלה: React בונה אתר כמו לגו. במקום דף אחד גדול, מחלקים לקוביות קטנות: Header, Post, Footer. כל קובייה יודעת להציג חלק קטן.

מה למדנו כאן?

- React היא ספריית צד לקוח לבניית ממשקי משתמש דינמיים.
- היא מבוססת JavaScript ונועדה לעבוד עם Components, כלומר רכיבים קטנים ונפרדים.
- Vite הוא כלי נוח ומהיר ליצירת פרויקט React חדש.
- index.html מכיל div עם id root. main.jsx מחבר את React לשורש הזה. App הוא הקומפוננטה הראשית.
- Component היא פונקציה שמחזירה JSX. JSX נראה כמו HTML בתוך JavaScript.
- ב-JSX חייבים להחזיר אלמנט אב אחד בלבד. class הופך ל-className. ביטויי JS נכתבים בתוך סוגריים מסולסלים {}.
- Props הם ערכים שהורה שולח לילד. למשל App שולח name ו-age לקומפוננטת Person.
- אפשר להציג אותה קומפוננטה כמה פעמים עם props שונים.
- CSS יכול להיות inline style או קובץ CSS שמייבאים לקומפוננטה.
- map משמש להצגת רשימה של קומפוננטות מתוך מערך. במקום לכתוב Post עשר פעמים, עוברים על המערך.
- התרגיל: ליצור מערכת פוסטים ב-Vite, קומפוננטת Post, מערך של 10 פוסטים, ובונוסים להוספת פוסט וחיבור ל-MongoDB.

מה חשוב לזכור?

- Component = קוביית לגו של ממשק.
- Props = מתנה שהורה נותן לילד.
- JSX = HTML בתוך JavaScript עם כמה חוקים מיוחדים.
- map = הדרך להציג רשימות בריאקט.
- App בדרך כלל מרכז את הקומפוננטות הראשיות.

דוגמה קטנה בקוד

```
function Post(props) {
  return (
    <div>
      <h2>{props.title}</h2>
      <p>{props.likes} likes</p>
    </div>
  );
}

{posts.map(post => <Post title={post.title} likes={post.likes} />)}
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן. חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

14. שיעורי בית 22 - React Basics, סטודנטים, Header, Dashboard, Footer ו-AddStudent

מקור: שיעורי בית 22.docx.pdf

אסוציאציה קלה: בונים לוח כיתה דיגיטלי: יש לוגו, טבלה של תלמידים, תחתית עם זכויות יוצרים, ואולי גם טופס שמוסיף תלמיד חדש.

מה למדנו כאן?

- פותחים פרויקט React חדש עם Vite ומנקים קבצים לא רלוונטיים מהתבנית.
- יוצרים קובץ students.js בתוך src/data ומייצאים ממנו מערך סטודנטים עם export default.
- ב-App מייבאים את המערך ומעבירים אותו לקומפוננטות.
- Header מציג לוגו בצד ימין.
- Dashboard מקבל את מערך הסטודנטים ומציג אותו כטבלה.
- Student יכולה להיות קומפוננטה שמייצגת שורה אחת או כרטיס אחד של סטודנט.
- Footer מציג משפט זכויות יוצרים ואת שם היוצר.
- בונוס AddStudent: קומפוננטה עם form שמאפשר להוסיף סטודנט חדש לטבלה.
- כדי להוסיף סטודנט חדש צריך להשתמש ב-useState, כי רשימת הסטודנטים משתנה בזמן שהאפליקציה עובדת.
- התרגיל מחזק ייבוא/ייצוא, קומפוננטות, map, props ו-state בסיסי.

מה חשוב לזכור?

- data folder עוזר להפריד בין נתונים לקומפוננטות.
- קומפוננטה שמקבלת מערך לרוב תציג אותו עם map.
- טופס שמוסיף מידע צריך state.
- לא משנים מערך state ישירות; יוצרים מערך חדש עם spread.

דוגמה קטנה בקוד

```
const [students, setStudents] = useState(initialStudents);

function addStudent(newStudent) {
  setStudents([...students, newStudent]);
}
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן.
חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

15. שיעור 22 - Hooks, useState, Immutable מול Mutable, onChange והעברת פונקציות ב-props

מקור: Lesson 22.pptx.pdf

> אסוציאציה קלה: useState הוא כמו לוח מחיק במטבח. כשהערך משתנה, React מסתכלת על הלוח ומציירת את המסך מחדש. אבל אסור למחוק את כל המחברת המקורית; עושים עותק חדש.

מה למדנו כאן?

- Hooks הן פונקציות מיוחדות של React. ההוק הראשון והחשוב הוא useState.
- useState שומר ערך שיכול להשתנות ומחזיר שני דברים: הערך הנוכחי ופונקציה שמעדכנת אותו.
- כאשר state משתנה, React מרנדרת מחדש את הקומפוננטה כדי להציג את הערך העדכני.
- Mutable code משנה את אותו אובייקט או מערך במקום. זה בעייתי בריאקט כי React משווה כתובות בזיכרון.
- Immutable code יוצר עותק חדש, ואז React יכולה לזהות שהשתנה משהו.
- מערכים ואובייקטים נשמרים לפי כתובת בזיכרון. לכן לא עושים push ישירות ל-state, אלא setList([...list, newItem]).
- onChange הוא אירוע שמופעל בכל שינוי באינפוט. בדרך כלל משתמשים בו כדי לעדכן state.
- לפעמים קומפוננטת ילד צריכה לשנות מידע שנמצא אצל ההורה. הפתרון: יוצרים פונקציה בהורה ושולחים אותה לילד דרך props.
- בדוגמת AddPost, App מחזיק את מערך הפוסטים ושולח פונקציות addPost ו-deletePost לילדים.
- לחיצה על כפתור או שינוי input מפעילים פונקציה שמשנה state, ואז React מציגה מחדש.

מה חשוב לזכור?

- useState = זיכרון משתנה בתוך קומפוננטה.

- setState = הדרך החוקית לעדכן state.
- לא דורסים state קיים, יוצרים עותק חדש.
- onChange מתאים לשדות קלט.
- פונקציה מהורה לילד מאפשרת לילד לבקש שינוי אצל ההורה.

דוגמה קטנה בקוד

```
const [title, setTitle] = useState('');
const [posts, setPosts] = useState([]);

function addPost(newPost) {
  setPosts([...posts, newPost]);
}

<input value={title} onChange={(e) => setTitle(e.target.value)} />
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן.
חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

16. שיעור 23 - React Router, Link, useNavigate, Context API, useContext ו-useParams

מקור: LESSON 23.pptx.pdf

אסוציאציה קלה: Router הוא כמו שלטים בבית: סלון, מטבח, חדר שינה. Link הוא שלט שלוחצים עליו. useNavigate הוא מפתח שמאפשר לעבור חדר רק אם התנאי מתאים. Context הוא מגש שמעביר דברים לכל החדרים בלי לעבור דרך כל האנשים בדרך.

מה למדנו כאן?

- Router מאפשר לשנות את כתובת ה-URL לפי העמוד שבו נמצאים, בלי לרענן את כל האתר.
- מתקינים react-router-dom ומייבאים BrowserRouter, Routes ו-Route.
- BrowserRouter עוטף את החלק באתר שמשתמש בניווט. Routes מכיל את כל הנתיבים. Route מגדיר נתיב וקומפוננטה.
- Link מאפשר מעבר בין עמודים דרך לחיצה, למשל <Link to="/about">About</Link>.
- useNavigate מאפשר ניווט מתוך פונקציה, למשל רק אחרי התחברות מוצלחת או אחרי בדיקת טופס.
- תכנון אתר: יש חלקים שלא קשורים לנתיבים, כמו Header קבוע; יש חלקים שקשורים לנתיבים אבל לא מתחלפים; ויש תוכן שמתחלף לפי הנתיב.
- Context API פותר בעיית props drilling, מצב שבו מעבירים props דרך קומפוננטות ביניים שלא באמת צריכות אותו.
- createContext יוצר הקשר. Provider עוטף את הקומפוננטות ומספק ערכים. useContext מושך את הערכים בקומפוננטה שצריכה אותם.
- useParams מאפשר לקרוא פרמטרים דינמיים מה-URL, למשל /user/:userId.
- התרגול כולל בניית אתר עם Home, About, Contact, תפריט משותף ובונוס לעיצוב אתר מותג בגדים.

מה חשוב לזכור?

- Route = איזה קומפוננטה מציגים באיזה נתיב.
- Link = מעבר פשוט בלחיצה.
- useNavigate = מעבר מתוך קוד ותנאים.
- Context = נתונים זמינים בהרבה קומפוננטות בלי שרשרת props.
- useParams = שליפת חלקים דינמיים מהכתובת.

דוגמה קטנה בקוד

```
<BrowserRouter>
  <Menu />
  <Routes>
    <Route path='/' element={<Home />} />
```

```
    <Route path='/user/:userId' element={<UserPage />} />
  </Routes>
</BrowserRouter>
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן. חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

17. שיעורי בית 23 - Smart House: חדרים, מוצרים, ניתוב ועיצוב עם AI

מקור: שיעורי בית 23.docx.pdf

> אסוציאציה קלה: האפליקציה היא בית חכם קטן. קודם מוסיפים חדרים, אחר כך נכנסים לכל חדר ומוסיפים מכשירים כמו מזגן, מנורה או סטריאו. כל חדר הוא כמו חדר אמיתי בבית.

מה למדנו כאן?

- צריך לבנות מערכת Smart House שמנוהלת על ידי משתמש: הוספת חדרים והוספת מכשירים לחדרים.
- חובה להשתמש ב-AI לעיצוב ולהעלות את התוצר ל-Vercel.
- מסך הבית הוא המסך הראשי ומציג את הכותרת smart house וכפתור הוספה.
- הוספת חדרים תהיה בערוץ /addroom ותכלול select ושני שדות טקסט.
- המשתמש בוחר סוג חדר, שם חדר וצבע. יש מגבלות כמו שם חדר עד 9 תווים, לפחות צבע אחד, וטיפול בשגיאות עם alert או הודעה.
- לאחר יצירת כמה חדרים, מסך הבית מציג ריבועים בצבעים שונים לפי החדרים שהמשתמש יצר.
- לחיצה על חדר פותחת עמוד חדר בערוץ /room. באתגר הנתיב דינמי לפי שם החדר, למשל /roomA או /roomeli.
- עמוד חדר מציג smart house, שם חדר וסוג חדר לפי הערכים שהמשתמש הכניס.
- בעמוד חדר יש אזור הוספת מוצר עם select של מוצרים כמו מזגן, מנורה, סטריאו, וכפתור הוסף.
- יש הגבלות: לא ניתן להוסיף אותו מוצר פעמיים באותו חדר, לחדר אמבטיה יש מכשיר מוגבל, ובכל חדר אפשר עד 5 מוצרים.
- לחיצה על מוצר משנה מצב בין דלוק לכבוי, למשל צבע/טקסט שמראה אם המוצר עובד.
- התרגיל מחבר React Router, state, ולידציות, רשימות דינמיות ועיצוב.

מה חשוב לזכור?

- כל חדר הוא אובייקט עם שם, סוג, צבע ומערך מוצרים.
- כל מוצר הוא אובייקט עם שם ומצב on/off.
- Router עוזר לעבור בין מסך בית, הוספת חדר ועמוד חדר.
- ולידציות חשובות כדי למנוע מצב לא הגיוני במערכת.
- AI יכול לעזור בעיצוב, אבל חוקי המערכת צריכים להיות בקוד שלכם.

דוגמה קטנה בקוד

```
const room = {
  name: 'Kitchen',
  type: 'Kitchen',
  color: 'green',
  devices: [{ name: 'Lamp', isOn: false }]
};
```

סיכום של סבתא

AI הוא כלי עזר מצוין, אבל המפתח נשאר אחראי. משתמשים בו כדי להתקדם מהר יותר, לא כדי לוותר על הבנה ובדיקה.

18. שיעור 24 - useEffect, Fetching Data, useMemo ו-useRef

מקור: lesson24.pptx.pdf

> אסוציאציה קלה: useEffect הוא כמו לומר: אחרי שהחדר צויר על המסך, תדליקי את האור או תביאי מידע מהמחסן. useRef הוא כמו פתק שמודבק לאלמנט במסך. useMemo הוא כמו לשמור תוצאה של חישוב

ארוך כדי לא לחשב שוב כל פעם.

מה למדנו כאן?

- useEffect הוא Hook שמבצע side effects, כלומר פעולות שקורות מחוץ לציור הרגיל של הקומפוננטה.
- דוגמאות ל-side effects: קריאת fetch לשרת, שינוי DOM ישיר, הגדרת טיימר.
- אם עושים fetch ישירות בתוך הקומפוננטה בלי useEffect, אפשר ליצור לולאה אינסופית: fetch מעדכן state, state גורם לרינדור, רינדור שולח fetch שוב.
- לכן עוטפים fetch ב-useEffect, ובדרך כלל מוסיפים dependency array ריק [] כדי שהקריאה תקרה פעם אחת בטעינת הקומפוננטה.
- useMemo שומר בזיכרון תוצאה של חישוב כבד, כדי שלא יחושב מחדש אם הנתונים הרלוונטיים לא השתנו.
- useRef יוצר אובייקט עם current. אפשר להשתמש בו כדי לגשת לאלמנט DOM, לשים פוקוס, לשמור ערך שלא גורם לרינדור, או לזכור מידע פנימי.
- התרגול כולל פרויקט React שמחכה 10 שניות וכותב HELLO WORLD, הוספת ראוטר עם About ו-Gallery, מעבר אוטומטי אחרי 10 שניות, ובונוס עם count וכפתור.

מה חשוב לזכור?

- useEffect = פעולה אחרי רינדור או בעקבות שינוי מסוים.
- fetch בריאקט שמים בתוך useEffect כדי לא ליצור לולאה.
- [] אומר: תפעיל פעם אחת אחרי טעינה ראשונה.
- useRef = גישה ישירה או שמירת ערך בלי רינדור.
- useMemo = זיכרון לחישוב יקר.

דוגמה קטנה בקוד

```
useEffect(() => {
  async function loadProducts() {
    const res = await fetch('https://dummyjson.com/products');
    const data = await res.json();
    setProducts(data.products);
  }
  loadProducts();
}, []);

const inputRef = useRef(null);
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

19. שיעורי בית 24 - אפליקציית סינון מוצרים עם useEffect, API ו-useRef

מקור: שיעורי בית 24.docx.pdf

אסוציאציה קלה: זה כמו חנות עם מדפים. בהתחלה מביאים את כל המוצרים מהמחסן, מציגים אותם, ואז נותנים ללקוח לבחור קטגוריה כדי לראות רק מדף מסוים.

מה למדנו כאן?

- צריך לבנות אפליקציית סינון מוצרים.
- האפליקציה חייבת לשלוף נתוני מוצרים מה-API: https://dummyjson.com/products.
- צריך להציג רשימת מוצרים ולפחות שם מוצר ומחיר. אפשר להוסיף גם תיאור, מותג וקטגוריה.
- צריך לספק select שמאפשר לסנן מוצרים לפי קטגוריה.
- תפריט הקטגוריות צריך להיווצר אוטומטית מהקטגוריות שהתקבלו מהמוצרים.
- חייבת להיות אפשרות הכל שמציגה את כל המוצרים.
- useEffect משמש לשליפת המוצרים כשהקומפוננטה נטענת.
- useRef משמש כדי להתמקד אוטומטית ב-select כשהדף נטען.
- המשימה מחזקת fetch, useEffect, state, סינון רשימות, select דינמי ו-useRef.

מה חשוב לזכור?

- נתונים מ-API נטענים ב-useEffect.
- קטגוריות לא כותבים ידנית אם אפשר לגזור אותן מהמוצרים.
- selectedCategory ב-state קובע מה להציג.
- useRef יכול לבצע focus על select או input.
- אופציית הכל מחזירה את הרשימה המקורית ללא סינון.

דוגמה קטנה בקוד

```
const filteredProducts = selectedCategory === 'all'
  ? products
  : products.filter(product => product.category === selectedCategory);
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

20. שיעור 25 - Tailwind CSS

מקור: Lesson 25.pptx.pdf

אסוציאציה קלה: Tailwind הוא כמו ארון בגדים עם מדבקות מוכנות: במקום לתפור חולצה מאפס, בוחרים מדבקות כמו רקע כחול, ריווח, פינות עגולות וטקסט ממורכז.

מה למדנו כאן?

- Tailwind CSS היא ספריית עיצוב פופולרית שעובדת עם utility classes.
- במקום לכתוב הרבה CSS בקובץ נפרד, מוסיפים classים קטנים ישירות לאלמנט ב-HTML או JSX.
- יתרונות: קל לשימוש, גמיש, מהיר, תומך ב-flex ו-grid, ומאפשר עיצוב מהיר בלי הרבה קבצי CSS.
- חסרונות: הקוד ב-HTML/JSX יכול להיראות עמוס בהרבה classים, ויש עקומת למידה.
- בהתקנה עם React ו-Vite משתמשים בהוראות Tailwind הרשמיות ומומלץ להתקין תוסף VS Code להשלמות.
- דוגמה: כדי ליצור כותרת עם רקע כחול, טקסט ממורכז, padding ופינות עגולות, משתמשים ב-classים מוכנים.
- Responsive design נעשה בעזרת prefixes כמו sm:, md:, lg: שמפעילים עיצוב לפי גודל מסך.
- המשימה: להתקין Tailwind בפרויקט חדש, ליצור div בסיסי עם רקע אדום, ובונוס לשנות צבע רקע בתצוגה קטנה.

מה חשוב לזכור?

- Tailwind = עיצוב בעזרת classים מוכנים.
- אין צורך לזכור הכל בעל פה; משתמשים בתיעוד וב-cheat sheet.
- Responsive prefixes עוזרים להתאים למסכים שונים.
- className בריאקט מכיל את מחלקות Tailwind.
- Tailwind לא מחליף חשיבה עיצובית; הוא רק כלי מהיר ליישום.

דוגמה קטנה בקוד

```
<div className='bg-red-500 md:bg-blue-300 p-4 rounded-lg text-center'>
  Hello Tailwind
</div>
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

21. פרויקט 25 - אתר דירוג סרטים

מקור: פרוייקט 25.docx.pdf

אסוציאציה קלה: זה כמו אתר המלצות לסרטים בבית: אפשר להוסיף סרט, לחפש סרט, למחוק סרט,

לדרג בכוכבים, ולראות מי הסרטים הכי אהובים.

מה למדנו כאן?

- מטרת האפליקציה: לאפשר למשתמש להוסיף סרטים, לדרג סרטים, להציג את שלושת הסרטים בדירוג הגבוה ביותר לפי ממוצע דירוגים, ולהציג חמישה סרטים רנדומליים בצד ימין.
- בחלק העליון יש כותרת קבועה Best Movie, ומתחתיה Nav Bar עם ארבעה כפתורים: בית, הוספה, מחיקה וחיפוש.
- כל סרט מוצג בקובייה שבה תמונת הסרט היא הרקע ושם הסרט במרכז.
- החלק הימני מציג רשימת חמישה סרטים רנדומליים, ממוינים לפי א-ב באנגלית מהאות הקטנה לגדולה.
- עמוד הבית מציג כברירת מחדל את הסרט הפופולרי ביותר במרכז.
- במרכז מוצגים movie name, תמונת הסרט, תיאור הסרט וסרגל דירוג של כוכב אחד עד חמישה כוכבים.
- לחיצה על דירוג מוסיפה את הדירוג לסרט, מעדכנת את ממוצע הדירוגים ומציגה אותו.
- לחיצה על קוביית סרט בחלק העליון או הימני משנה את הסרט שמוצג במרכז.
- עמוד הוספת סרט כולל כותרת, שני inputים, textarea לתיאור וכפתור הוספה.
- שם הסרט חייב להיות באנגלית. גם אם המשתמש כתב באותיות גדולות/קטנות בצורה לא אחידה, יש להמיר לכיתוב תקין בסגנון Title Case.
- תיאור הסרט חייב להיות באנגלית ועד 200 תווים. תמונה אינה חובה.
- בלחיצה על Add Movie מציגים התראה מתאימה אם הסרט נוצר או אם יש שגיאות משתמש.
- עמוד חיפוש כולל input וכפתור חיפוש. החיפוש לא חייב לקבל שם מלא; אם יש כמה תוצאות, מציגים למשתמש בחירה, למשל select.
- עמוד מחיקה מקבל שם מלא של סרט, לא חלקי. אם נמצא, מוחק; אם לא, מתריע שאין סרט כזה.
- הפרויקט מחבר state, רשימות, חיפוש, מחיקה, ולידציות, דירוגים, חישוב ממוצע, עיצוב וניווט.

מה חשוב לזכור?

- סרט הוא אובייקט עם name, image, description ו-ratings.
- ממוצע דירוגים = סכום הדירוגים חלקי מספר הדירוגים.
- חיפוש חלקי יכול להחזיר כמה תוצאות, ולכן צריך טיפול במצב של יותר מתוצאה אחת.
- מחיקה דורשת שם מלא כדי למנוע מחיקה בטעות.
- הפרויקט דורש גם חוויית משתמש, לא רק לוגיקה.

דוגמה קטנה בקוד

```
function getAverageRating(movie) {
  if (movie.ratings.length === 0) return 0;
  const sum = movie.ratings.reduce((total, rating) => total + rating, 0);
  return sum / movie.ratings.length;
}
```

סיכום של סבתא

JavaScript הוא הבסיס: משתנים מחזיקים מידע, תנאים מקבלים החלטות, לולאות עוברות על רשימות, פונקציות מסדרות קוד, וה-DOM מחבר את הקוד למסך.

TypeScript עם React-ו AI, TypeScript

22. שימוש נכון ב-AI בפיתוח

מקור: שימוש בAI בצורה נכונה בפיתוח.pdf

אסוציאציה קלה: AI הוא כמו עוזר חכם במטבח. הוא יכול להציע מתכון, לקצוץ ירקות ולעזור מהר, אבל את עדיין הטבחית. את בודקת, מחליטה ולא מגישה אוכל שלא טעמת.

מה למדנו כאן?

- השיעור מבדיל בין שימוש ב-AI כחלק ממערכת לבין שימוש ב-AI לפרודוקטיביות של מפתחים.
- כלים למפתחים כוללים Cursor, Windsurf ו-Claude Code.
- כלים מונגשים למפתחים כוללים V0, Bolt.new ו-GitHub Spark.
- כלים ל"לא מפתחים" כוללים Base44, Lovable ו-A0.dev.
- כלים שהם ממש ל"לא מפתחים" כוללים FlutterFlow ו-Bubble.io.
- שימוש חכם ב-AI דורש ניהול גרסאות נכון, למשל Git ו-GitHub, כדי שאפשר לחזור אחורה אם משהו נשבר.
- צריך לוודא כל שורת קוד. AI יכול לטעות, להמציא או ליצור קוד שלא מתאים לפרויקט.
- לא לרוץ אחרי הכלי אלא לנהל אותו: לתת הוראות ברורות, לבדוק תוצאה, לשאול למה הקוד עובד.
- לא להתחייב למנויים שנתיים בלי צורך. להשתמש בכלי כשצריך, לנצל מה שיש בחינם או בתשלום בצורה חכמה, ולהיות מוכנים להחליף כלי.
- התרגול: לבחור כלי AI ולהתנסות בו.

מה חשוב לזכור?

- AI הוא עוזר, לא תחליף להבנה.
- כל קוד ש-AI כותב צריך בדיקה.
- Git לפני ניסויים גדולים.
- בקשו הסבר מה-AI, לא רק קוד.
- השתמשו ב-AI גם לעיצוב, רעיונות ושיפור טקסטים, לא רק להעתקת פתרונות.

סיכום של סבתא

AI הוא כלי עזר מצוין, אבל המפתח נשאר אחראי. משתמשים בו כדי להתקדם מהר יותר, לא כדי לוותר על הבנה ובדיקה.

23. שיעור 26 - TypeScript בסיסי: טיפוסים, Compiler, tsconfig, Tuple, Enum, Functions, Objects ו-Type

מקור: Lesson 26.pptx.pdf

אסוציאציה קלה: TypeScript הוא JavaScript עם תוויות על הקופסאות. אם הקופסה כתוב עליה "מספר", אי אפשר לשים בה פתאום מחרוזת. זה מונע בלגן לפני שמפעילים את הקוד.

מה למדנו כאן?

- TypeScript היא שפה שבנויה מעל JavaScript ומוסיפה טיפוסים חזקים יותר.
- בניגוד ל-JavaScript, שבה משתנה יכול לשנות סוג במהלך החיים, ב-TypeScript מגדירים מה סוג המשתנה.
- היתרון: מגלים טעויות לפני הרצת הקוד. העורך והקומפיילר מזהירים כשמשהו לא מתאים.
- דפדפנים לא קוראים TypeScript ישירות. צריך לבצע compile או transpile ל-JavaScript בעזרת tsc.
- tsconfig.json הוא קובץ הגדרות שקובע איך TypeScript יתנהג בפרויקט.
- Tuple הוא מערך באורך קבוע שבו לכל מקום יש type משלו, למשל [id, name].
- Enum מתאים לקבוצה קבועה של אפשרויות, למשל מידות בגדים או ז'אנרים.
- בפונקציות מגדירים type לכל פרמטר וגם type של הערך שמוחזר. אם אין החזרה, משתמשים ב-void.
- באובייקטים מגדירים מראש את סוג כל מאפיין. readonly אומר שלא ניתן לשנות ערך. סימן ? אומר שהשדה אופציונלי.
- Type מאפשר לתת שם למבנה נתונים ולמחזר אותו בקוד, במקום לכתוב את אותה הגדרה שוב ושוב.

- המשימות כוללות משתנה שם, מערך מספרים, פונקציית סכום, מכוניות עם פרמטרים, Enum לדגם, ו-Tuple עם כמה סוגי ערכים.

מה חשוב לזכור?

- TypeScript לא מחליף JavaScript; הוא מוסיף בדיקות וסדר.
- number, string, boolean הם טיפוסים בסיסיים.
- void = פונקציה שלא מחזירה ערך.
- type מתאים להגדרת מבנה שנשתמש בו שוב.
- Enum מתאים לאפשרויות קבועות מראש.

דוגמה קטנה בקוד

```
type Car = {
  name: string;
  color: string;
  year: number;
  isLeasing: boolean;
};

function sum(numbers: number[]): number {
  return numbers.reduce((total, num) => total + num, 0);
}
```

סיכום של סבתא

TypeScript מוסיף תוויות לקוד. הוא לא עושה קסמים, אבל הוא מזהיר מראש כששמים דבר לא מתאים במקום הלא נכון, וזה חוסך הרבה תקלות.

24. שיעור 26 חלק 2 - TypeScript עם React: Props ו-State

מקור: Lesson 26 part 2.pptx.pdf

אסוציאציה קלה: בריאקט רגיל שולחים לילד קופסה ואומרים "תסתדר". בריאקט עם TypeScript מדביקים על הקופסה תווית: יש בה טקסט, מספר, ופונקציה שלא מחזירה כלום.

מה למדנו כאן?

- שימוש ב-TypeScript עם React נותן Type Safety, כלומר פחות הפתעות בזמן ריצה.
- קל יותר לעשות Debugging כי טעויות רבות נתפסות כבר בזמן כתיבה.
- חוויית המפתח טובה יותר: השלמות קוד, אזהרות והבנה ברורה מה כל קומפוננטה מצפה לקבל.
- נהוג להגדיר את Type של props בתחילת קובץ הקומפוננטה כדי לשמור על קריאות.
- כאשר שולחים פונקציה מהורה לילד, צריך לציין גם את סוג הפרמטרים וגם מה הפונקציה מחזירה. אם היא לא מחזירה כלום, כותבים void.
- State גם צריך type. אפשר להגדיר useState<number>, useState<string> או useState<Todo>[] לפי הצורך.
- נהוג לשים types או models בקבצים חיצוניים, למשל models/Todo.ts, כדי שהמבנה יהיה מסודר וניתן לשימוש חוזר.
- הדוגמה המרכזית היא Todo: מגדירים איך נראית משימה אחת, ואז משתמשים במערך של משימות ב-state.

מה חשוב לזכור?

- Props type מגדיר מה קומפוננטה מקבלת.
- פונקציה שלא מחזירה כלום = () => void או (id: number) => void.
- Model חיצוני עוזר לשמור על קוד נקי.
- useState<Todo>[] אומר שה-state הוא מערך של Todo.
- TypeScript עוזר במיוחד בפרויקטים גדולים עם הרבה קומפוננטות.

דוגמה קטנה בקוד

```
type Todo = {
  id: number;
  title: string;
  completed: boolean;
};

type TodoProps = {
  todo: Todo;
  onDelete: (id: number) => void;
};

const [todos, setTodos] = useState<Todo[]>([]);
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן.
חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

25. שיעורי בית TS 1 - ניהול ספרים וניהול פרופיל משתמש

מקור: lesson 27_homwork.docx.pdf / שיעורי בית TS 1

> אסוציאציה קלה: זה כמו ספרייה קטנה וכרטיסי משתמשים. כל ספר חייב כרטיס מסודר, וכל משתמש יכול להיות אורח או משתמש רשום. TypeScript דואג שלא נערבב ביניהם.

מה למדנו כאן?

- חלק ראשון: ניהול ספרים. מגדירים type בשם Book עם title, author, publishedDate, available ו-genre.
- genre נבנה בעזרת enum עם לפחות חמישה ז׳אנרים, למשל Fiction, NonFiction, Biography, ScienceFiction ו-Fantasy.
- כותבים פונקציה addBook שמקבלת מערך ספרים ואובייקט ספר, ומוסיפה ספר חדש.
- כותבים פונקציה searchByGenre שמקבלת מערך ספרים ו-genre ומחזירה ספרים שמתאימים לז׳אנר.
- כותבים updateAvailability שמעדכנת זמינות של ספר לפי title ו-available.
- חלק שני: ניהול פרופיל משתמש. מגדירים interface בשם BaseUser עם id, username ו-email.
- מרחיבים את BaseUser לשני ממשקים: GuestUser עם guestSessionId, ו-RegisteredUser עם profile ו-lastLogin.
- יוצרים Type בשם User שיכול להיות GuestUser או RegisteredUser.
- displayUserInfo מקבלת User ומדפיסה מידע שונה לפי סוג המשתמש. צריך להשתמש ב- type narrowing בצורה נכונה.
- updateUserProfile מקבלת User ואובייקט עדכון פרופיל, אבל צריכה לעבוד רק על RegisteredUser.
- התרגיל מחזק enum, type, interface, union types, function typing ו-type guards.

מה חשוב לזכור?

- type מתאים למבנה נתונים. interface מתאים במיוחד לאובייקטים שנרצה להרחיב.
- enum מתאים לרשימת ערכים קבועה.
- Union type אומר: המשתנה יכול להיות אחד מכמה סוגים.
- Type narrowing אומר לבדוק איזה סוג באמת קיבלנו לפני שניגשים לשדות מיוחדים.
- לא לעדכן profile למשתמש אורח שאין לו profile.

דוגמה קטנה בקוד

```
enum Genre {
  Fiction,
  NonFiction,
  Biography,
  ScienceFiction,
  Fantasy
}
```

```
type Book = {
  title: string;
  author: string;
  publishedDate: string;
  available: boolean;
  genre: Genre;
};
```

סיכום של סבתא

TypeScript מוסיף תוויות לקוד. הוא לא עושה קסמים, אבל הוא מזהיר מראש כששמים דבר לא מתאים במקום הלא נכון, וזה חוסך הרבה תקלות.

26. שיעורי בית React + TypeScript 2 - אפליקציית ניהול תקציב

מקור: שיעורי בת React + TS 2.docx.pdf

> אסוציאציה קלה: זה כמו מחברת תקציב ביתית: רושמים כל הכנסה וכל הוצאה, רואים כמה נכנס, כמה יצא, כמה נשאר, ואיפה הכסף הלך.

מה למדנו כאן?

- צריך לפתח אפליקציה ב-React + TypeScript + Tailwind לניהול תקציב.
- האפליקציה מאפשרת למשתמשים לעקוב אחר הכנסות והוצאות.
- משתמשים יכולים להוסיף טרנזקציה עם כמות, סוג טרנזקציה - הכנסה או הוצאה, קטגוריה, תיאור ותאריך.
- משתמשים יכולים לצפות ברשימת כל הטרנזקציות בסדר כרונולוגי הפוך, כלומר החדשות קודם.
- משתמשים יכולים לערוך את תיאור ההכנסה או ההוצאה ולמחוק טרנזקציות.
- תקציר התקציב צריך להציג סך הכנסות, סך הוצאות וסך חסכונות לפי כל הטרנזקציות שהוכנסו.
- הסיכומים צריכים להתעדכן בזמן אמת כאשר טרנזקציות נוספות, נערכות או נמחקות.
- צריך להציג פירוט הוצאות לפי קטגוריות ולהראות כמה בוזבז בכל קטגוריה.
- TypeScript עוזר להגדיר Transaction מסודר, Tailwind עוזר לעצב במהירות, ו-React state מנהל את הנתונים המשתנים.
- הפרויקט מחבר כמעט את כל החומר: טפסים, state, map/filter/reduce, עריכה, מחיקה, חישובים, TypeScript ו-UI.

מה חשוב לזכור?

- Transaction הוא אובייקט מרכזי עם id, amount, type, category, description ו-date.
- סך חסכונות = סך הכנסות פחות סך הוצאות.
- reduce מתאים לחישוב סכומים.
- filter מתאים להצגת הוצאות בלבד או קטגוריה מסוימת.
- State צריך להתעדכן בצורה immutable כדי שהמסך יתעדכן.

דוגמה קטנה בקוד

```
type Transaction = {
  id: number;
  amount: number;
  type: 'income' | 'expense';
  category: string;
  description: string;
  date: string;
};

const totalIncome = transactions
  .filter(t => t.type === 'income')
  .reduce((sum, t) => sum + t.amount, 0);
```

סיכום של סבתא

React עוזרת לבנות מסך מחלקים קטנים. כל חלק מקבל מידע, מציג אותו, וכשמידע משתנה המסך מתעדכן.
חשוב לשמור על סדר: state במקום הנכון, props למידע שעובר לילדים, ו-router לעמודים.

מילון קטן למושגים שחוזרים הרבה

מושג	הסבר פשוט
API	דרך שבה מערכת אחת מדברת עם מערכת אחרת ומבקשת מידע או שולחת מידע.
CRUD	ארבע פעולות - Create, Read, Update, Delete בסיסיות על מידע: ליצור, לקרוא, לעדכן ולמחוק.
State	מידע שמשתנה בזמן שהאפליקציה עובדת. בריאקט שינוי state גורם לרינדור מחדש.
Props	מידע שהורה שולח לקומפוננטת ילד בריאקט.
Hook	פונקציה מיוחדת של React, למשל useState או useEffect.
DOM	המבנה של דף ה-HTML כפי שהדפדפן רואה אותו, כדי ש-JavaScript יוכל לשנות את המסך.
Promise	הבטחה לתוצאה עתידית - הצלחה או כישלון.
fetch	פקודה לשליחת בקשה לשרת וקבלת נתונים.
Middleware	פונקציית ביניים בשרת שבודקת או משנה בקשה לפני שהיא ממשיכה.
Schema	תבנית שמגדירה איך אמור להיראות מידע במסד נתונים.
Type	הגדרה ב-TypeScript שמסבירה איזה סוג מידע מותר לשים במשתנה או אובייקט.
Enum	רשימה קבועה של אפשרויות, כמו סוגי ספרים או גדלי חולצות.

מקורות שנכללו במסמך

- Lesson 11.pptx.pdf
- שיעורי בית 11.docx.pdf
- שיעורי בית 12.docx.pdf
- Lesson 13.pptx.pdf
- שיעורי בית 13.docx.pdf
- lesson 15.pptx.pdf
- חוברת תרגול JavaScript.pdf
- lesson 19.pdf
- Lesson 16.pptx.pdf
- Lesson 17.pptx.pdf
- שיעורי בית 18.docx.pdf
- Lesson 20.pptx.pdf
- Lesson 21.pptx.pdf
- שיעורי בית 22.docx.pdf
- Lesson 22.pptx.pdf
- LESSON 23.pptx.pdf
- שיעורי בית 23.docx.pdf
- lesson24.pptx.pdf

- שיעורי בית 24.docx.pdf
- Lesson 25.pptx.pdf
- פרוייקט 25.docx.pdf
- שימוש בAI בצורה נכונה בפיתוח.pdf
- Lesson 26.pptx.pdf
- Lesson 26 part 2.pptx.pdf
- שיעורי בית 1 TS / lesson 27_homwork.docx.pdf
- שיעורי בת React + TS 2.docx.pdf